

Question 1) Implement Quick Sort using two different partitioning schemes Lomuto and Hoare and to analyze and compare their performance in terms of execution time and number of swaps performed, and to justify which scheme provides better efficiency.

1. Algorithm:

Here's the algorithm explained in plain wording (no pseudocode):

Quick Sort :

1. If the portion of the array being looked at has more than one element, pick a pivot element from it.
2. Rearrange (partition) the elements so that everything smaller than the pivot ends up on its left, and everything larger ends up on its right. The exact method of doing this rearrangement is where Lomuto and Hoare differ.
3. Once partitioned, repeat the same process separately on the left portion and the right portion.
4. Keep repeating this until every portion has just one element left, at which point the whole array is sorted.

Lomuto Partition Scheme :

1. Choose the last element of the current portion as the pivot.
2. Keep a marker (call it the "boundary") that tracks the last position known to hold a value smaller than or equal to the pivot. Start this boundary just before the portion begins.
3. Scan through the portion from left to right, one element at a time.
4. Whenever the current element is found to be smaller than or equal to the pivot, move the boundary one step forward and swap the current element into that boundary position.
5. After scanning the entire portion, swap the pivot (which is still at the end) into the position right after the boundary. This places the pivot in its correct final sorted position.
6. The position where the pivot now sits divides the array into a left part (all smaller elements) and a right part (all larger elements).

Hoare Partition Scheme :

1. Choose the first element of the current portion as the pivot.
2. Set up two pointers: one starting just before the beginning of the portion, and one starting just after the end.
3. Move the left pointer rightward, step by step, until it lands on an element that is greater than or equal to the pivot.

4. Move the right pointer leftward, step by step, until it lands on an element that is less than or equal to the pivot.
5. If the left pointer has now reached or passed the right pointer, stop — the right pointer's current position is returned as the dividing point.
6. Otherwise, swap the elements at the left and right pointers, and repeat the process from step 3.
7. Unlike Lomuto, the pivot does not necessarily end up in its final sorted position — the returned dividing point simply splits the portion into two sides that get sorted further.

1.2 Lomuto Partition

LOMUTO-PARTITION(arr, low, high)

```
1. pivot <- arr[high]
2. i <- low - 1
3. for j <- low to high - 1
4.   if arr[j] <= pivot
5.     i <- i + 1
6.     swap arr[i], arr[j]
7.   end if
8. end for
9. swap arr[i + 1], arr[high]
10. return i + 1
```

1.3 Hoare Partition

HOARE-PARTITION(arr, low, high)

```
1. pivot <- arr[low]
2. i <- low - 1
3. j <- high + 1
4. loop
5.   repeat i <- i + 1 until arr[i] >= pivot
6.   repeat j <- j - 1 until arr[j] <= pivot
7.   if i >= j return j
8.   swap arr[i], arr[j]
9. end loop
```

Note: For Hoare partition, the recursive calls in QUICKSORT become QUICKSORT(arr, low, pi) and QUICKSORT(arr, pi + 1, high), since Hoare's scheme does not guarantee the pivot lands at its final sorted index.

3. Full Program (C)

The following program implements both partitioning schemes, counts swaps performed by each, generates random test arrays of increasing size, and measures execution time using clock().

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

long long swapCountLomuto = 0;
long long swapCountHoare = 0;

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

/* ----- Lomuto Partition Scheme ----- */
int lomutoPartition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
            swapCountLomuto++;
        }
    }
    swap(&arr[i + 1], &arr[high]);
    swapCountLomuto++;
    return i + 1;
}

void quickSortLomuto(int arr[], int low, int high) {
    if (low < high) {
        int pi = lomutoPartition(arr, low, high);
        quickSortLomuto(arr, low, pi - 1);
        quickSortLomuto(arr, pi + 1, high);
    }
}
```

```

/* ----- Hoare Partition Scheme ----- */
int hoarePartition(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low - 1;
    int j = high + 1;
    while (1) {
        do { i++; } while (arr[i] < pivot);
        do { j--; } while (arr[j] > pivot);
        if (i >= j) return j;
        swap(&arr[i], &arr[j]);
        swapCountHoare++;
    }
}

void quickSortHoare(int arr[], int low, int high) {
    if (low < high) {
        int pi = hoarePartition(arr, low, high);
        quickSortHoare(arr, low, pi);
        quickSortHoare(arr, pi + 1, high);
    }
}

int isSorted(int arr[], int n) {
    for (int i = 1; i < n; i++)
        if (arr[i - 1] > arr[i]) return 0;
    return 1;
}

int* generateRandomArray(int n, unsigned int seed) {
    int *arr = malloc(n * sizeof(int));
    srand(seed);
    for (int i = 0; i < n; i++) arr[i] = rand() % 100000;
    return arr;
}

void runTrial(int n, unsigned int seed) {
    int *base = generateRandomArray(n, seed);

    int *arr1 = malloc(n * sizeof(int));
    memcpy(arr1, base, n * sizeof(int));
    swapCountLomuto = 0;
    clock_t start1 = clock();

```

```

quickSortLomuto(arr1, 0, n - 1);
clock_t end1 = clock();
double timeLomuto = ((double)(end1 - start1)) / CLOCKS_PER_SEC * 1000.0;

int *arr2 = malloc(n * sizeof(int));
memcpy(arr2, base, n * sizeof(int));
swapCountHoare = 0;
clock_t start2 = clock();
quickSortHoare(arr2, 0, n - 1);
clock_t end2 = clock();
double timeHoare = ((double)(end2 - start2)) / CLOCKS_PER_SEC * 1000.0;

printf("%-10d | %-9s | %-12lld | %-10.4f\n", n, "Lomuto", swapCountLomuto,
timeLomuto);
printf("%-10d | %-9s | %-12lld | %-10.4f\n", n, "Hoare", swapCountHoare, timeHoare);

if (!isSorted(arr1, n) || !isSorted(arr2, n)) {
    printf("ERROR: array not sorted correctly for n=%d\n", n);
}

free(base); free(arr1); free(arr2);
}

int main() {
    int sizes[] = {1000, 5000, 10000, 20000, 40000};
    int numSizes = sizeof(sizes) / sizeof(sizes[0]);

    printf("%-10s | %-9s | %-12s | %-10s\n", "Size(n)", "Scheme", "Swaps", "Time(ms)");
    printf("-----\n");

    for (int i = 0; i < numSizes; i++) {
        runTrial(sizes[i], 42);
    }

    return 0;
}

```

ASSESSMENT-CO3/AT1-EXPERIMENTS

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Quick Sort
10 marks



1/1 answered

Q1 Quick Sort

10 marks

Analyze the performance of Quick Sort under different partitioning schemes such as Lomuto and Hoare. Record execution time and number of swaps. Compare results and interpret differences. Justify which partitioning scheme provides better efficiency.

Your Code

```
1 #include<stdio.h>
2 #include<time.h>
3 int swapsL=0,swapsH=0;
4 void swap(int *a,int *b)
5 {
6     int temp=*a;
7     *a=*b;
8     *b=temp;
9 }
10 int lomutoPartition(int arr[],int low,int high)
11 {
12     int pivot = arr[high];
13     int i = low -1;
14     for(int j=low;j<high;j++)
15     {
16         if(arr[j]<=pivot)
17         {
18             swap(&arr[i+1],&arr[j]);
19             swapsL++;
20             return i+1;
21         }
22     }
23     return high;
24 }
25 void quickSortLomuto(int arr[],int low,int high)
26 {
27     if(low<high)
28     {
29         int pi=lomutoPartition(arr,low,high);
30         quickSortLomuto(arr,low,pi-1);
31         quickSortLomuto(arr,pi+1,high);
32     }
33 }
34 int hoarePartition(int arr[],int low,int high)
35 {
36     int pivot=arr[low];
37     int i=low-1;
38     int j=high+1;
39     while(1)
40     {
41         do{
42             i++;
43         }while(arr[i]<pivot);
44         do{
45             j--;
46         }while(arr[j]>pivot);
47         if(i>=j)
48             return j;
49         swap(&arr[i],&arr[j]);
50         swapsH++;
51     }
52 }
53 void quickSortHoare(int arr[],int low,int high)
54 {
55     if(low<high)
56     {
57         int pi=hoarePartition(arr,low,high);
58         quickSortHoare(arr,low,pi-1);
59         quickSortHoare(arr,pi+1,high);
60     }
61 }
62 int main()
63 {
64     int n;
65     printf("Enter number of elements:");
66     scanf("%d",&n);
67     int arrL[n],arrH[n];
68     printf("Enter elements for Lomuto partitioning scheme:");
69     for(int i=0;i<n;i++)
70         scanf("%d",&arrL[i]);
71     printf("Enter elements for Hoare partitioning scheme:");
72     for(int i=0;i<n;i++)
73         scanf("%d",&arrH[i]);
74     clock_t start,end;
75     double timeL,timeH;
76     start = clock();
77     quickSortLomuto(arrL,0,n-1);
78     end = clock();
79     timeL=(long double)(end-start)/CLOCKS_PER_SEC;
80     printf("\nLomuto Sorted Array:");
81     for(int i=0;i<n;i++)
82         printf("%d ",arrL[i]);
83     printf("\n");
84     printf("\nLomuto Swaps:%d",swapsL);
85     printf("\nLomuto Swaps:%f seconds\n",timeL);
86     start = clock();
87     quickSortHoare(arrH,0,n-1);
88     end = clock();
89     timeH=(long double)(end-start)/CLOCKS_PER_SEC;
90     printf("\nHoare Sorted Array:");
91     for(int i=0;i<n;i++)
92         printf("%d ",arrH[i]);
93     printf("\n");
94     printf("\nHoare swaps:%d",swapsH);
95     printf("\nHoare swaps:%f seconds\n",timeH);
96     if(swapsH<swapsL)
97     {
98         printf("\nHoare partition is more efficient because it performs fewer swaps than Lomuto partitioning scheme.\n");
99     }
100 }
```

Input

10
9 7 10 5 8 1 6 2 3 4

Output

enter number of elements:enter
elements:
Lomuto Sorted Array:12345678910
Lomuto Swaps:16
Lomuto Swaps:0.000000 seconds